I. Unit Testing Principles và JUnit 5

2.1. Tổng quan về Software Testing

Software Testing là quá trình kiểm tra phần mềm nhằm phát hiện lỗi, xác minh hành vi và đánh giá mức độ đáp ứng yêu cầu.

Kiểm thử không chỉ nhằm xác nhận chương trình chạy được, mà còn giúp phát hiện các trường hợp đầu vào không hợp lệ, lỗi xử lý dữ liệu và những hành vi không đúng với yêu cầu nghiệp vụ.

Một số cấp độ kiểm thử phổ biến:

Cấp độ	Mục đích	Ví dụ
Unit Testing	Kiểm tra một đơn vị nhỏ độc lập	Kiểm tra phương thức tính tổng
Integration Testing	Kiểm tra sự phối hợp giữa các thành phần	Service kết hợp Repository và database
System Testing	Kiểm tra toàn bộ hệ thống	Kiểm tra quy trình đặt hàng hoàn chỉnh
Acceptance Testing	Xác minh hệ thống đáp ứng yêu cầu người dùng	Kiểm tra các kịch bản nghiệp vụ được chấp nhận

Các cấp độ này có phạm vi khác nhau. Unit test thường chạy nhanh và dễ xác định nguyên nhân lỗi; integration test kiểm tra tương tác thật giữa các thành phần; system và acceptance test đánh giá hành vi ở phạm vi rộng hơn.

2.2. Unit Testing là gì?

Unit Testing là phương pháp kiểm thử một đơn vị nhỏ của chương trình, chẳng hạn một phương thức hoặc một class, nhằm xác minh rằng nó hoạt động đúng trong những điều kiện xác định.

Ví dụ, với phương thức:

```
public int add(int a, int b) {
    return a + b;
}
```

Một unit test có thể kiểm tra kết quả khi đầu vào là 2 và 3, đồng thời kiểm tra trường hợp số âm hoặc các giá trị biên phù hợp với yêu cầu.

2.2.1. Lợi ích của Unit Testing

- Phát hiện lỗi sớm trong quá trình phát triển.
- Giảm nguy cơ lỗi khi sửa đổi hoặc bổ sung chức năng.

- Giúp xác định rõ hành vi mong đợi của từng thành phần.
- Hỗ trợ refactoring an toàn hơn.
- Cung cấp ví dụ thực thi về cách sử dụng một class hoặc phương thức.

2.2.2. Nguyên tắc viết Unit Test

1. Kiểm thử hành vi cụ thể

Mỗi test nên có mục tiêu rõ ràng, chẳng hạn kiểm tra kết quả tính toán hoặc xác minh một ngoại lệ được ném ra.

2. Độc lập

Một test không nên phụ thuộc vào việc test khác đã chạy trước hay chưa. Trạng thái cần thiết phải được khởi tạo trong chính test hoặc trong phần thiết lập chung phù hợp.

3. Có thể lặp lại

Test nên tạo ra kết quả ổn định khi chạy nhiều lần trong cùng điều kiện. Tránh phụ thuộc không kiểm soát vào thời gian thực, mạng hoặc dữ liệu bên ngoài.

4. Dễ đọc và dễ bảo trì

Tên test nên thể hiện hành vi đang kiểm tra. Phần chuẩn bị dữ liệu, thực hiện hành động và xác minh kết quả cần được phân tách rõ ràng.

5. Kiểm tra cả trường hợp hợp lệ và không hợp lệ

Không chỉ kiểm tra trường hợp chương trình hoạt động đúng trong điều kiện thông thường mà còn phải xem xét dữ liệu rỗng, dữ liệu không hợp lệ, giá trị biên và các lỗi có thể xảy ra.

6. Không phụ thuộc vào chi tiết triển khai không cần thiết

Test nên ưu tiên xác minh hành vi và kết quả có ý nghĩa với người sử dụng hoặc thành phần gọi đến, thay vì kiểm tra quá nhiều chi tiết nội bộ không thuộc hợp đồng của class.

2.3. Nguyên tắc AAA

AAA là cách tổ chức một bài kiểm thử thành ba phần:

- **Arrange:** Chuẩn bị dữ liệu và đối tượng.
- **Act:** Thực hiện hành động cần kiểm tra.
- **Assert:** Xác minh kết quả thực tế có đúng với mong đợi hay không.

Ví dụ:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
```

```
import org.junit.jupiter.api.Test;


class CalculatorTest {


    @Test
    void addTwoNumbers_returnsSum() {
        // Arrange
        Calculator calculator = new Calculator();


        // Act
        int result = calculator.add(2, 3);


        // Assert
        assertEquals(5, result);
    }
}
```

Ở ví dụ này, Calculator là đối tượng được kiểm thử, add(2, 3) là hành động và assertEquals(5, result) xác minh kết quả.

AAA không phải một quy tắc cú pháp bắt buộc của JUnit mà là nguyên tắc giúp test dễ đọc và dễ hiểu.

2.4. JUnit 5

JUnit 5 là framework kiểm thử phổ biến trong Java. JUnit 5 gồm ba thành phần chính:

- **JUnit Platform:** Nền tảng để khởi chạy các framework kiểm thử.
- **JUnit Jupiter:** Cung cấp mô hình lập trình và các annotation cho JUnit 5.
- **JUnit Vintage:** Hỗ trợ chạy một số test viết theo JUnit 3 và JUnit 4 trên nền tảng JUnit tương ứng.

Khi viết test mới với JUnit 5, thông thường người học sử dụng JUnit Jupiter.

2.4.1. Các annotation quan trọng

Annotation	Chức năng

@Test	Đánh dấu một phương thức kiểm thử
@BeforeEach	Chạy trước mỗi test method
@AfterEach	Chạy sau mỗi test method
@BeforeAll	Chạy một lần trước tất cả test trong class hoặc container tương ứng
@AfterAll	Chạy một lần sau tất cả test trong class hoặc container tương ứng
@DisplayName	Đặt tên dễ đọc cho test
@ParameterizedTest	Chạy cùng một test với nhiều bộ dữ liệu
@RepeatedTest	Lặp lại một test một số lần
@Disabled	Tạm bỏ qua test

Mặc định, @BeforeAll và @AfterAll thường được khai báo là static trong mô hình vòng đời mặc định của JUnit Jupiter. Có thể dùng phương thức instance nếu thay đổi @TestInstance sang PER_CLASS.

2.4.2. Các assertion thường gặp

Assertion là các phương thức dùng để xác minh điều kiện mong đợi trong test.

Assertion	**Mục đích**
assertEquals(expected, actual)	Kiểm tra hai giá trị bằng nhau
assertNotEquals(a, b)	Kiểm tra hai giá trị không bằng nhau
assertTrue(condition)	Kiểm tra điều kiện đúng
assertFalse(condition)	Kiểm tra điều kiện sai
assertNull(value)	Kiểm tra giá trị là null
assertNotNull(value)	Kiểm tra giá trị khác null
assertThrows(Exception.class, ...)	Kiểm tra hành động ném ngoại lệ đúng loại
assertAll(...)	Thực hiện nhiều assertion trong cùng một nhóm

Ví dụ:

```
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;
```

```
class ValidationTest {

    @Test
    void validatesInput() {
        String name = "Duy";

        assertNotNull(name);
        assertEquals("Duy", name);
        assertTrue(name.length() > 0);
    }
}
```

2.4.3. Kiểm tra ngoại lệ

Giả sử phương thức yêu cầu số lượng phải lớn hơn 0:

```
public class QuantityValidator {

    public void validate(int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException(
                    "Quantity must be positive");
        }
    }
}
```

Test:

```
import static org.junit.jupiter.api.Assertions.assertThrows;
import org.junit.jupiter.api.Test;

class QuantityValidatorTest {
```

```
    @Test
    void validate_whenQuantityIsZero_throwsException() {
        QuantityValidator validator = new QuantityValidator();

        assertThrows(
                IllegalArgumentException.class,
                () -> validator.validate(0)
        );
    }
}
```

Bài test xác minh rằng khi truyền số lượng bằng 0, phương thức ném IllegalArgumentException.

Nếu cần kiểm tra thêm nội dung thông báo, có thể lưu ngoại lệ được trả về từ assertThrows() rồi so sánh getMessage() với giá trị mong đợi.

2.4.4. Parameterized Test

Parameterized Test cho phép chạy cùng một logic kiểm thử với nhiều bộ dữ liệu khác nhau.

Ví dụ:

```
import static org.junit.jupiter.api.Assertions.assertTrue;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

class PositiveNumberTest {

    @ParameterizedTest
    @ValueSource(ints = {1, 2, 10, 100})
    void positiveNumbers_areGreaterThanZero(int number) {
        assertTrue(number > 0);
    }
}
```

JUnit sẽ chạy test với từng giá trị được khai báo.

Parameterized Test giúp giảm việc lặp lại code khi muốn kiểm tra nhiều đầu vào có cùng mục tiêu.

2.4.5. Phân biệt các loại test

- **Positive test:** Kiểm tra đầu vào hợp lệ và hành vi mong đợi.
- **Negative test:** Kiểm tra đầu vào không hợp lệ hoặc tình huống lỗi.
- **Boundary test:** Kiểm tra các giá trị biên như 0, giá trị tối thiểu và tối đa.
- **Regression test:** Kiểm tra lại các hành vi đã hoạt động đúng sau khi code được thay đổi để phát hiện lỗi phát sinh.

Một bộ unit test tốt thường kết hợp nhiều loại test phù hợp với yêu cầu nghiệp vụ.

2.5. Ví dụ Unit Testing hoàn chỉnh

Giả sử có class Calculator:

```
public class Calculator {


    public int add(int a, int b) {
        return a + b;
    }


    public int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException(
                    "Divisor cannot be zero");
        }


        return a / b;
    }
}
```

Test:

```
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.BeforeEach;
```

```
import org.junit.jupiter.api.Test;

class CalculatorTest {

    private Calculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new Calculator();
    }

    @Test
    void add_returnsCorrectSum() {
        assertEquals(5, calculator.add(2, 3));
    }

    @Test
    void add_supportsNegativeNumbers() {
        assertEquals(-3, calculator.add(-1, -2));
    }

    @Test
    void divide_returnsQuotient() {
        assertEquals(2, calculator.divide(10, 5));
    }

    @Test
    void divide_byZero_throwsException() {
        assertThrows(
```

```
            IllegalArgumentException.class,
            () -> calculator.divide(10, 0)
        );
    }
}
```

Bộ test trên kiểm tra các trường hợp tính toán bình thường, số âm và phép chia có mẫu số bằng 0. Để đánh giá kỹ hơn, có thể bổ sung các giá trị biên và trường hợp tràn số nếu đó là yêu cầu của ứng dụng.

III. Mocking với Mockito

3.1. Mocking là gì?

Mocking là kỹ thuật sử dụng các đối tượng giả lập để thay thế dependency thật trong một bài kiểm thử.

Mock thường được cấu hình để trả về dữ liệu mong muốn khi một phương thức được gọi, hoặc để xác minh rằng một tương tác cụ thể đã diễn ra.

Mục tiêu chính là cô lập thành phần đang được kiểm thử, tránh phụ thuộc vào database, API bên ngoài, email server hoặc các hệ thống khác khi chạy unit test.

Ví dụ, khi kiểm thử UserService, không nhất thiết phải kết nối database thật. Có thể sử dụng một mock của UserRepository để kiểm soát kết quả truy vấn.

3.2. Phân biệt Mock, Stub, Fake và Spy

Khái niệm	Đặc điểm
Mock	Đối tượng giả lập thường dùng để cấu hình và xác minh tương tác
Stub	Cung cấp dữ liệu trả về cố định cho các lời gọi
Fake	Một implementation đơn giản nhưng có hành vi thực tế ở mức cơ bản, ví dụ repository lưu dữ liệu trong bộ nhớ
Spy	Theo dõi và có thể cấu hình một đối tượng thật, tùy cách sử dụng

Các khái niệm này có thể chồng lấn trong thực tế và không phải framework nào cũng phân biệt hoàn toàn giống nhau.

3.3. Mockito là gì?

Mockito là framework phổ biến trong Java để tạo mock, cấu hình hành vi giả lập và xác minh tương tác giữa các đối tượng.

Các API thường gặp:

- @Mock: khai báo một mock.
- @InjectMocks: yêu cầu Mockito tạo đối tượng cần kiểm thử và đưa các mock phù hợp vào các dependency.
- when(...).thenReturn(...): cấu hình kết quả trả về.
- verify(...): xác minh một phương thức đã được gọi.
- never(): xác minh phương thức không được gọi.
- doThrow(...): cấu hình một mock ném ngoại lệ trong những trường hợp phù hợp.

3.4. Ví dụ kiểm thử UserService

Giả sử có interface repository:

```
import java.util.Optional;


public interface UserRepository {
    Optional<User> findById(Long id);
    User save(User user);
}
```

Class User:

```
public class User {
    private Long id;
    private String name;


    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }


    public Long getId() {
        return id;
    }
```

```
    public String getName() {
        return name;
    }
}
```

Service cần kiểm thử:

```
import java.util.NoSuchElementException;

public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User getUser(Long id) {
        return userRepository.findById(id)
                .orElseThrow(() ->
                        new NoSuchElementException(
                                "User not found"));
    }
}
```

Ở đây, UserService phụ thuộc vào UserRepository thông qua constructor. Đây là constructor injection, giúp việc thay thế dependency trong test trở nên đơn giản.

3.4.1. Viết unit test với Mockito

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import java.util.Optional;
```

```
import java.util.NoSuchElementException;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void getUser_whenUserExists_returnsUser() {
        // Arrange
        User user = new User(1L, "Duy");

        when(userRepository.findById(1L))
                .thenReturn(Optional.of(user));

        // Act
        User result = userService.getUser(1L);

        // Assert
```

```
        assertEquals("Duy", result.getName());

        verify(userRepository).findById(1L);
    }

    @Test
    void getUser_whenUserDoesNotExist_throwsException() {
        when(userRepository.findById(2L))
                .thenReturn(Optional.empty());

        assertThrows(
                NoSuchElementException.class,
                () -> userService.getUser(2L)
        );

        verify(userRepository).findById(2L);
    }
}
```

3.4.2. Giải thích cách hoạt động

Bước 1: Tạo mock repository

@Mock tạo một đối tượng giả lập của UserRepository. Đối tượng này không truy vấn database thật.

Bước 2: Khởi tạo service

@InjectMocks yêu cầu Mockito tạo UserService và cung cấp mock phù hợp cho constructor của nó.

Bước 3: Cấu hình hành vi

Lệnh:

```
when(userRepository.findById(1L))
        .thenReturn(Optional.of(user));
```

quy định rằng khi mock nhận lời gọi findById(1L), nó trả về Optional chứa đối tượng user.

Bước 4: Thực hiện và kiểm tra

Test gọi userService.getUser(1L) và kiểm tra kết quả.

Bước 5: Xác minh tương tác

verify(userRepository).findById(1L) xác nhận service đã gọi repository với ID mong muốn.

3.5. Stub và Verify khác nhau thế nào?

Stub xác định mock sẽ trả về gì:

```
when(repository.findById(1L))
        .thenReturn(Optional.of(user));
```

Verify xác định tương tác nào đã xảy ra:

```
verify(repository).findById(1L);
```

Không nên lạm dụng verify() cho mọi lời gọi nội bộ. Nên dùng khi tương tác đó quan trọng với hành vi đang kiểm thử, chẳng hạn xác nhận service không gọi cổng thanh toán khi đơn hàng không hợp lệ.

3.6. Mocking và Dependency Injection

Dependency Injection (DI) là cách cung cấp dependency cho một đối tượng từ bên ngoài thay vì để đối tượng tự khởi tạo tất cả dependency của nó.

Ví dụ:

```
public class OrderService {


    private final PaymentGateway paymentGateway;


    public OrderService(PaymentGateway paymentGateway) {
        this.paymentGateway = paymentGateway;
    }
}
```

Trong ứng dụng thực tế, PaymentGateway có thể là implementation kết nối cổng thanh toán. Trong unit test, có thể truyền mock vào để kiểm soát phản hồi và mô phỏng lỗi.

DI giúp thay thế dependency dễ dàng và làm cho code có khả năng kiểm thử tốt hơn.

3.7. Những lỗi thường gặp khi dùng Mockito

- Quên khởi tạo Mockito extension hoặc cơ chế mở rộng tương ứng.
- Cấu hình mock không khớp với đối số thực tế được truyền vào.
- Mock quá nhiều dependency khiến test chỉ kiểm tra tương tác giả lập mà không kiểm tra được logic đáng kể.
- Dùng verify() quá chi tiết khiến test dễ hỏng khi refactor.
- Nhầm rằng unit test có mock repository là đã kiểm tra được câu SQL hoặc mapping database.

Nguyên tắc: Chỉ mock những dependency cần thiết để cô lập đơn vị đang kiểm thử. Với hành vi truy vấn database, mapping entity hoặc transaction, cần bổ sung integration test phù hợp.

IV. Integration Testing Strategies

4.1. Integration Testing là gì?

Integration Testing là quá trình kiểm thử sự phối hợp giữa nhiều thành phần của phần mềm.

Khác với unit test, vốn thường thay dependency bằng mock, integration test tập trung xác minh các thành phần thực sự tương tác đúng với nhau.

Ví dụ, trong một ứng dụng Spring Boot:

Controller

|

v

Service

|

v

Repository

|

v

Database

Một unit test của Service có thể mock Repository để kiểm tra logic nghiệp vụ. Một integration test có thể sử dụng database kiểm thử để xác minh truy vấn, mapping entity và thao tác lưu dữ liệu.

4.2. Vì sao cần Integration Testing?

Một hệ thống có thể vượt qua toàn bộ unit test nhưng vẫn gặp lỗi khi tích hợp vì:

- Mapping entity không đúng với schema database.
- Câu truy vấn hoạt động khác dự kiến trên database thật.
- Cấu hình dependency injection bị thiếu hoặc sai.
- JSON request/response không được chuyển đổi đúng.
- Transaction hoặc constraint của database gây lỗi.
- Các thành phần dùng giao diện hoặc kiểu dữ liệu không tương thích.

Integration test giúp phát hiện các lỗi này mà unit test đơn thuần có thể không tìm ra.

4.3. Các chiến lược Integration Testing

4.3.1. Big Bang Integration Testing

Tất cả hoặc hầu hết thành phần được tích hợp cùng lúc rồi mới kiểm thử.

Ưu điểm:

- Cách tổ chức ban đầu tương đối đơn giản.
- Có thể phù hợp với hệ thống nhỏ.

Hạn chế:

- Khó xác định thành phần gây lỗi.
- Nhiều lỗi tích hợp có thể xuất hiện cùng lúc.
- Thường khó kiểm soát với hệ thống lớn.

4.3.2. Top-Down Integration Testing

Bắt đầu từ các thành phần cấp cao như Controller hoặc module điều phối, sau đó tích hợp dần các thành phần cấp thấp hơn.

Khi dependency cấp thấp chưa sẵn sàng, có thể sử dụng stub để mô phỏng hành vi của chúng.

Ưu điểm:

- Có thể kiểm tra luồng điều khiển và kiến trúc cấp cao sớm.
- Giúp phát hiện lỗi ở các thành phần điều phối.

Hạn chế:

- Cần tạo stub cho các dependency chưa sẵn sàng.
- Các chức năng cấp thấp được kiểm tra đầy đủ muộn hơn.

4.3.3. Bottom-Up Integration Testing

Bắt đầu từ các thành phần cấp thấp như repository, database adapter hoặc thư viện nghiệp vụ rồi tích hợp dần lên các thành phần cấp cao hơn.

Ưu điểm:

- Có thể xác minh các thành phần nền tảng trước.
- Hữu ích khi logic cấp thấp có vai trò quan trọng.

Hạn chế:

- Cần driver hoặc thành phần gọi thử nếu module cấp cao chưa sẵn sàng.
- Luồng nghiệp vụ tổng thể được kiểm tra muộn hơn.

4.3.4. Incremental Integration Testing

Tích hợp từng phần theo từng bước, kiểm thử sau mỗi lần thêm thành phần.

Đây thường là cách dễ quản lý hơn trong dự án phát triển hiện đại vì giúp khoanh vùng lỗi tốt hơn so với việc tích hợp tất cả cùng lúc.

4.3.5. So sánh các chiến lược

Chiến lược	Cách thực hiện	Hạn chế chính
Big Bang	Tích hợp nhiều thành phần cùng lúc	Khó tìm nguyên nhân lỗi
Top-Down	Từ cấp cao xuống cấp thấp	Cần stub
Bottom-Up	Từ cấp thấp lên cấp cao	Có thể cần driver
Incremental	Tích hợp và kiểm thử từng phần	Cần lập kế hoạch thứ tự tích hợp

4.4. Integration Testing trong Spring Boot

Spring Boot cung cấp các công cụ hỗ trợ kiểm thử thành phần được quản lý bởi Spring Container.

Một số annotation phổ biến:

Annotation	Mục đích
@SpringBootTest	Khởi tạo ApplicationContext để kiểm thử với cấu hình Spring rộng hơn

@WebMvcTest	Tập trung kiểm thử tầng Spring MVC
@DataJpaTest	Tập trung kiểm thử các thành phần JPA
@AutoConfigureMockMvc	Cấu hình MockMvc khi sử dụng trong test context phù hợp
@Transactional	Quản lý transaction cho test hoặc phương thức theo cấu hình tương ứng

Cần chọn annotation dựa trên phạm vi cần kiểm thử. Không phải test nào cũng cần khởi động toàn bộ ApplicationContext.

4.5. Ví dụ kiểm thử Repository với @DataJpaTest

Giả sử có entity:

```
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    protected Product() {
    }

    public Product(String name) {
        this.name = name;
```

```
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

Repository:

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository
        extends JpaRepository<Product, Long> {
}
```

Integration test:

```
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

@DataJpaTest
class ProductRepositoryTest {

    @Autowired
    private ProductRepository productRepository;
```

```
    @Test
    void saveProduct_persistsProduct() {
        Product product = new Product("Laptop");

        Product saved = productRepository.save(product);

        assertNotNull(saved.getId());
        assertEquals("Laptop", saved.getName());
    }
}
```

Giải thích

- @DataJpaTest khởi tạo cấu hình kiểm thử tập trung vào JPA.
- Repository được cung cấp bởi Spring Container.
- Bài test lưu entity và kiểm tra dữ liệu trả về.

Theo cấu hình mặc định thông dụng, @DataJpaTest chạy trong transaction được rollback sau test. Database sử dụng có thể là database nhúng nếu dependency phù hợp có mặt, hoặc một cấu hình database kiểm thử khác. Vì vậy, cần xác nhận database thực tế mà bài test đang dùng.

Bài test này kiểm tra việc lưu entity trong phạm vi JPA, nhưng chưa đủ để chứng minh toàn bộ quy trình tạo sản phẩm qua HTTP hoạt động đúng.

4.6. Kiểm thử tầng Web với MockMvc

MockMvc cho phép gửi request đến các thành phần Spring MVC trong môi trường test mà không cần mở một server HTTP thực sự.

Ví dụ minh họa:

```
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;

import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;


import org.junit.jupiter.api.Test;

import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(ProductController.class)
class ProductControllerTest {

    @Autowired
    private MockMvc mockMvc;

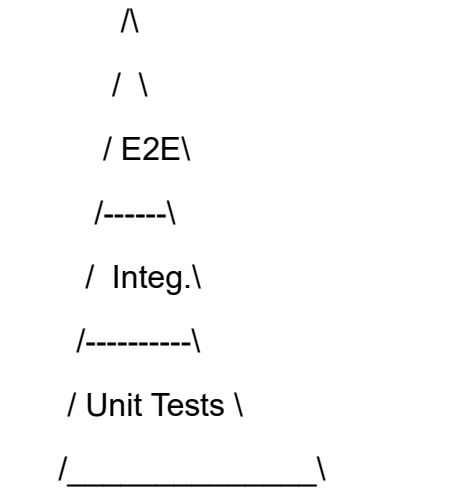
    @Test
    void getProducts_returnsOk() throws Exception {
        mockMvc.perform(get("/products"))
                .andExpect(status().isOk());
    }
}
```

Ví dụ này giả định ProductController có endpoint GET /products và cấu hình để trả về HTTP 200 trong tình huống kiểm thử. Nếu controller có dependency service, cần cung cấp dependency đó bằng mock hoặc cấu hình phù hợp.

MockMvc giúp kiểm tra routing, status code và chuyển đổi request/response ở tầng MVC, nhưng không tự động kiểm tra toàn bộ hệ thống qua mạng HTTP thật.

4.7. Test Pyramid

Test Pyramid là mô hình định hướng phân bổ các loại kiểm thử:



Ý tưởng chung:

- Unit test thường chiếm số lượng lớn vì nhanh và dễ khoanh vùng lỗi.
- Integration test xác minh các thành phần tương tác với nhau.
- End-to-end test kiểm tra những luồng nghiệp vụ hoàn chỉnh, thường tốn chi phí hơn.

Đây là nguyên tắc định hướng, không phải tỷ lệ bắt buộc cho mọi dự án. Cơ cấu kiểm thử cần dựa vào kiến trúc, rủi ro và chi phí bảo trì của hệ thống.

V. Test Coverage và Quality Metrics

5.1. Test Coverage là gì?

Test Coverage là nhóm chỉ số dùng để đo lường mức độ mà bộ kiểm thử đã thực thi hoặc bao phủ các phần của chương trình.

Coverage cao không đồng nghĩa với việc chương trình không có lỗi. Một dòng code có thể đã chạy qua nhưng chưa được kiểm tra kết quả đúng hoặc chưa được kiểm tra trong mọi tình huống quan trọng.

5.2. Line Coverage

Line Coverage đo tỷ lệ dòng code có thể thực thi đã được chạy trong quá trình kiểm thử, theo cách công cụ đo lường định nghĩa.

Công thức khái quát:

$$
\text{Line Coverage} =
\frac{\text{Số dòng thực thi đã chạy}}
{\text{Tổng số dòng thực thi được đo}}
\times 100\%
$$

Ví dụ, nếu có 100 dòng thực thi và test chạy qua 80 dòng thì Line Coverage là 80%.

Chỉ số này giúp nhận biết những khu vực chưa được thực thi, nhưng không chứng minh các kết quả trên những dòng đó đã được xác minh chính xác.

5.3. Statement Coverage

Statement Coverage đo tỷ lệ câu lệnh thực thi đã được chạy.

Về mặt thực tế, Statement Coverage và Line Coverage có thể có giá trị khác nhau tùy cách code được trình bày và cách công cụ đếm câu lệnh. Ví dụ, một dòng có thể chứa nhiều câu lệnh, hoặc một câu lệnh kéo dài trên nhiều dòng.

5.4. Branch Coverage

Branch Coverage đo mức độ bao phủ các nhánh quyết định, chẳng hạn hai kết quả đúng và sai của một điều kiện if.

Ví dụ:

```
public String checkAge(int age) {
    if (age >= 18) {
        return "Adult";
    }

    return "Minor";
}
```

Nếu test chỉ gọi checkAge(20), chương trình đã đi theo nhánh điều kiện đúng nhưng chưa kiểm tra nhánh sai.

Để bao phủ cả hai nhánh, cần kiểm thử cả age >= 18 và age < 18.

Công thức khái quát:

$$
\text{Branch Coverage} = \frac{\text{Số nhánh đã thực thi}}{\text{Tổng số nhánh được đo}} \times 100\%
$$

5.5. Condition Coverage

Condition Coverage đo việc từng điều kiện con trong một biểu thức logic nhận các giá trị đúng và sai.

Ví dụ:

```
if (age >= 18 && hasId) {
    return "Allowed";
}
```

Có hai điều kiện con:

- age >= 18
- hasId

Một bộ test cần được thiết kế để các điều kiện con nhận cả giá trị true và false.

Condition Coverage khác Branch Coverage. Đạt một loại coverage không nhất thiết đồng nghĩa với việc đạt loại còn lại, đặc biệt với biểu thức có nhiều điều kiện.

5.6. Method Coverage

Method Coverage đo tỷ lệ phương thức được gọi trong quá trình kiểm thử, theo định nghĩa của công cụ.

Chỉ số này hữu ích để nhận diện các phương thức chưa được thực thi, nhưng một phương thức được gọi một lần không có nghĩa tất cả nhánh hoặc trường hợp biên của nó đã được kiểm tra.

5.7. So sánh các loại Coverage

Loại	Đo lường	Điểm hạn chế
Line Coverage	Dòng code được thực thi	Không thể hiện đầy đủ các nhánh
Statement Coverage	Câu lệnh được thực thi	Không chứng minh logic được kiểm tra đủ
Branch Coverage	Các nhánh quyết định	Không bảo đảm mọi tổ hợp điều kiện được kiểm tra
Condition Coverage	Giá trị đúng/sai của điều kiện con	Không bảo đảm mọi nhánh hoặc tổ hợp đều được kiểm tra
Method Coverage	Phương thức đã được gọi	Không phản ánh mức độ kiểm thử bên trong phương thức

5.8. Ví dụ tính Coverage

Giả sử một class có 20 dòng thực thi, trong đó 16 dòng được chạy qua:

$$
\text{Line Coverage} = \frac{16}{20}\times100\%=80\%
$$

Giả sử có 10 nhánh cần đo và 7 nhánh đã được thực thi:

$$
\text{Branch Coverage} = \frac{7}{10}\times100\%=70\%
$$

Kết quả cho thấy bộ test đã thực thi 80% số dòng và 70% số nhánh được đo. Đây không phải bằng chứng rằng chất lượng phần mềm đạt lần lượt 80% và 70%; vẫn cần xem xét assertion, dữ liệu kiểm thử, lỗi phát hiện được và rủi ro nghiệp vụ.

5.9. Công cụ đo Coverage: JaCoCo

JaCoCo là công cụ phổ biến trong hệ sinh thái Java để đo code coverage khi chạy test.

JaCoCo có thể cung cấp báo cáo về:

- Instruction Coverage.
- Branch Coverage.
- Line Coverage.
- Method Coverage.
- Class Coverage.
- Complexity-related counters.

JaCoCo thường được tích hợp với Maven hoặc Gradle và có thể tạo báo cáo HTML để người phát triển xem các phần code đã được bao phủ hoặc chưa được bao phủ.

Một báo cáo Coverage nên được sử dụng để tìm các khu vực có rủi ro, chứ không nên chỉ chạy theo một con số phần trăm duy nhất.

5.10. Quality Metrics

Ngoài Coverage, có nhiều chỉ số được sử dụng để đánh giá chất lượng phần mềm và hiệu quả kiểm thử.

5.10.1. Pass Rate

Pass Rate đo tỷ lệ test đạt trong tổng số test được xem xét.

$$
\text{Pass Rate} =
\frac{\text{Số test Passed}}
{\text{Tổng số test được thực thi}}
\times 100\%
$$

Ví dụ: 95 test đạt trên tổng số 100 test được thực thi thì Pass Rate là 95%.

Cần phân biệt test thất bại với test bị bỏ qua hoặc không được thực thi. Các báo cáo thực tế có thể sử dụng mẫu số khác nhau, vì vậy cần ghi rõ quy ước.

5.10.2. Defect Density

Defect Density đo mật độ lỗi trong một đơn vị phần mềm, thường được tính trên số dòng code hoặc kích thước phần mềm.

Một công thức minh họa:

$$
\text{Defect Density} =

\frac{\text{Số lỗi được ghi nhận}}
{\text{Kích thước phần mềm}}
]

Ví dụ, 10 lỗi trên 1.000 dòng code tương ứng 10 lỗi/KSLOC, nếu KSLOC được quy ước là 1.000 dòng code.

Chỉ số này phụ thuộc vào cách định nghĩa lỗi, phạm vi sản phẩm, thời gian ghi nhận và cách đo kích thước. Không nên so sánh trực tiếp các dự án nếu phương pháp đo khác nhau.

5.10.3. Defect Detection Rate

Defect Detection Rate có thể được sử dụng để mô tả tốc độ hoặc tỷ lệ phát hiện lỗi trong một giai đoạn kiểm thử. Cần định nghĩa rõ công thức, thời gian và mẫu số vì các tổ chức có thể sử dụng cách tính khác nhau.

Ví dụ, có thể thống kê số lỗi được phát hiện trong mỗi vòng kiểm thử để đánh giá xu hướng phát hiện lỗi.

5.10.4. Defect Leakage

Defect Leakage mô tả lỗi lọt qua một giai đoạn kiểm thử và chỉ được phát hiện ở giai đoạn sau, chẳng hạn lỗi chỉ xuất hiện khi người dùng kiểm thử hoặc khi phần mềm đã được triển khai.

Một công thức tham khảo:

[
\text{Defect Leakage Rate} =
\frac{\text{Lỗi phát hiện ở giai đoạn sau}}
{\text{Tổng lỗi được tính trong phạm vi}}
\times 100%
]

Mẫu số phải được định nghĩa rõ. Ví dụ, nếu tính lỗi phát hiện sau khi phát hành chia cho tổng lỗi trước và sau phát hành, cần áp dụng cùng phạm vi sản phẩm và khoảng thời gian.

5.10.5. Mutation Testing

Mutation Testing đánh giá khả năng phát hiện lỗi của bộ test bằng cách cố ý tạo ra những thay đổi nhỏ trong code, chẳng hạn đổi toán tử > thành >=, rồi chạy lại test.

- **Killed mutant:** bộ test phát hiện sự thay đổi và thất bại như mong đợi.
- **Survived mutant:** bộ test vẫn vượt qua dù code đã bị thay đổi.
- **Equivalent mutant:** thay đổi không làm thay đổi hành vi quan sát được trong phạm vi đang xét.

Mutation Testing có thể phát hiện những bộ test có Coverage cao nhưng assertion yếu. Tuy nhiên, nó tốn thời gian hơn việc đo Coverage thông thường.

5.10.6. Test Execution Time

Test Execution Time là thời gian cần thiết để chạy bộ test hoặc một nhóm test.

Test quá chậm có thể làm giảm tần suất chạy trong quá trình phát triển và CI/CD. Có thể cải thiện bằng cách:

- Tăng số lượng unit test nhanh.
- Hạn chế khởi động các dependency không cần thiết.
- Tách unit test và integration test thành các nhóm.
- Chạy song song khi phù hợp.
- Tối ưu việc khởi tạo database hoặc ApplicationContext.

5.10.7. Tổng hợp Quality Metrics

Chỉ số	Ý nghĩa
Line Coverage	Mức bao phủ dòng code
Branch Coverage	Mức bao phủ nhánh điều kiện
Pass Rate	Tỷ lệ test đạt
Defect Density	Mật độ lỗi trên đơn vị kích thước
Defect Leakage	Mức lỗi lọt qua giai đoạn kiểm thử
Mutation Score	Khả năng phát hiện các mutation có ý nghĩa
Test Execution Time	Thời gian chạy test

Không có một chỉ số đơn lẻ nào đủ để kết luận phần mềm có chất lượng tốt. Cần kết hợp độ bao phủ, chất lượng assertion, kết quả kiểm thử, lỗi thực tế, hiệu năng và mức độ đáp ứng yêu cầu.